DAY 11: ADVANCED PYTHON AUTOMATION & COMPUTATIONAL NOTES

Discrete Variables (PMF) & Continuous Variables (PDF) Manual Engine Analytics

Bhai, aapke bataye mutabik footer margin configurations aur header accent border layers ko sateek tarike se update kar diya hai. Har page ke left corner par aapka naam permanently register ho gaya hai.

1. DISCRETE RANDOM VARIABLE (PMF) - ANALYSIS

Discrete variables me data countable points me hota hai. Hamara main core logic pehle manual iteration engine se chalega, phir use single-line vectorized array slicing se replace karenge.

A. Manual Step-by-Step Code

```
# ======================================================================
# DAY 11: DISCRETE RANDOM VARIABLE (PMF) - MANUAL ENGINE
# ======================================================================

x_values = [0, 1, 2, 3, 4]  # X = Number of luxury jewelry orders per hour
pmf_probabilities = [0.40, 0.25, 0.11, 0.16, 0.08]  # Calculated via k=0.08

# --- STAGE 1: MANUAL AXIOM VALIDATION CHECK ---
total_pmf_sum = 0.0
is_valid_pmf = True

# Manual loop chalakar har ek value ko check aur sum kar rahe hain
for i in range(len(pmf_probabilities)):
    current_prob = pmf_probabilities[i]

    # Check A: Koi bhi probability negative (< 0) nahi honi chahiye
    if current_prob < 0:
        is_valid_pmf = False

    total_pmf_sum += current_prob

print(f"-> Calculated Total Sum of PMF Box = {total_pmf_sum:.2f}")

# --- STAGE 2: TARGET PROBABILITY EXECUTION [P(X >= 3)] ---
target_probability = 0.0

for i in range(len(x_values)):
    current_x = x_values[i]

    # Condition Trigger: Jab hamara sateek number 'x' >= 3 hoga, tabhi sum karenge
    if current_x >= 3:
        target_probability += pmf_probabilities[i]
        print(f"-> Target Condition Matched! Included: P(X = {current_x}) =
{pmf_probabilities[i]:.2f}")
```

```
final_percentage = target_probability * 100
print(f"FINAL OUTCOME: {target_probability:.2f} (or {final_percentage:.2f}%)")
```

> 🔥 **Core Breakdowns & Doubts Clarification:**
>
> **1. Baseline Variables Matrix:** `total_pmf_sum = 0.0` ek empty bucket ki tarah kaam karta hai jisme data elements accumulate hote hain. `is_valid_pmf = True` ek security flag keeper hai jo kisi bhi invalid check par structural state ko shift karta hai.
>
> **2. Index Pointer Mechanics:** `range(len(pmf_probabilities))` se computer ko absolute sequence allocation `[0, 1, 2, 3, 4]` milta hai. Loop ka pointer `i` isi numeric order ko sequentially track karta hai.
>
> **3. Target Accumulator Layer:** `if current_x >= 3:` logic dynamic index filter ka kaam karta hai. Jis specific point par index rule match hota hai, theek usi index code `i` ka value extracted hokar process hota hai.
>
> **4. Automated Carriage Return Logic:** Python ka print engine background me default parameter *`end="\n"`* execute karta hai. Is wajah se har standard loop iteration apna task execute karne ke baad automatic line separator lagakar screen space optimize karti hai.

B. Production Short Code (NumPy Vectorization)

```
import numpy as np

x_values = np.array([0, 1, 2, 3, 4])
pmf_probabilities = np.array([0.40, 0.25, 0.11, 0.16, 0.08])

# Shortcut Code: Single Line Array Condition Masking & Summation
target_probability = np.sum(pmf_probabilities[x_values >= 3])
print(f"P(X >= 3) Short Output: {target_probability * 100:.2f}%")
```

> 🧠 **NumPy Vectorization Execution Trace:**
>
> Yahan background data engine me linear scanning loops complete bypass ho jate hain. `x_values >= 3` direct memory sequence me Boolean Mask generate karta hai: `[False, False, False, True, True]`. Yeh filter layer jab arrays ke bracket interface me integrate hoti hai, toh automatic targeting matrix execution direct computational values stream out kar deta hai.

2. CONTINUOUS RANDOM VARIABLE (PDF) - ANALYSIS

Continuous random variables me system data points structural limits ke mathematical curve ke form me set hote hain. Hamara production scenario hai: $f(y) = 0.5y$ jahan execution scale limit $0 \le y \le 2$ hai.

A. Pure Mathematical Execution Matrix

Stage 1: Constant Rule Validation

Total area space hamesha absolute execution identity constant `1` hona chahiye:

$$\int_0^2 c \cdot y \, dy = 1 \Rightarrow c \cdot [\, y^2 / 2 \,]_0^2 = 1 \Rightarrow c \cdot (4/2 - 0) = 1 \Rightarrow 2c = 1 \Rightarrow c = 0.5$$

Stage 2: Target Window Probability Estimation

Manager ka structural specification target calculation filter nikalna hai for $P(1 \le Y \le 2)$:

$$P(1 \le Y \le 2) = \int_1^2 0.5y \, dy = 0.5 \cdot [\, y^2 / 2 \,]_1^2 = 0.5 \cdot (4/2 - 1/2) = 0.5 \cdot 1.5 = 0.75 \ (75.00\%)$$

B. Continuous Variable Manual Loop Engine

```
# ======================================================================
# DAY 11: CONTINUOUS RANDOM VARIABLE (PDF) - MANUAL ENGINE
# ======================================================================

c_constant = 0.5  # Derived mathematical scaling factor
dy = 0.00001      # Infinitesimal differential width unit

# --- STAGE 1: TOTAL AREA INTEGRATION OVER BOUNDARY [0.0 TO 2.0] ---
total_area = 0.0
current_y = 0.0

while current_y < 2.0:
    height = c_constant * current_y   # Current coordinate functional altitude
    tiny_area = height * dy           # Area calculation of micro element
    total_area += tiny_area           # Systematic accumulation
    current_y += dy                   # Boundary precision displacement

print(f"-> Calculated Total Area under the curve = {total_area:.4f}")

# --- STAGE 2: TARGET WINDOW AREA ESTIMATION [1.0 TO 2.0] ---
target_range_area = 0.0
current_y = 1.0   # Directly initializing index counter at lower integration node

while current_y < 2.0:
    height = c_constant * current_y
    tiny_area = height * dy
    target_range_area += tiny_area
    current_y += dy

print(f"FINAL TARGET PROBABILITY RESULT = {target_range_area * 100:.2f}%")
```

🛠️ High-Precision Loop Iteration Mechanism Check:

1. Step Size & Total Rounds Math: Total absolute iteration loops count engine structure total step calculation rule follow karta hai: ***Distance / Step Size = (2.0 - 0.0) / 0.00001 = 200,000 rounds*** (Exact 2 Lakh iterations execution process setup). Computer exactly 2 Lakh times continuous loop logic stream execute karega aur processing parameters save honge.

2. Dynamic Differentiation & Integration Properties: ***dy*** execution logic standalone absolute function entity nahi hai, balki coordinate changes scale step structure hai. Differentiation component processing me data chunks break hote hain aur integration engine un steps ko parallel slice tracking variable mechanism se sequential scale par accumulative manner me save karta jata hai.

C. Processing Simulation Data Trace (With Step Tracking Scale `dy = 0.5`)

Computer structural runtime processing logic execution sequence numbers step mapping matrix framework is data trace table pattern ko follow karta hai:

Step Index	Track Coordinate (y)	Altitude Height: f(y) = 0.5 * y	Element Slice Area: Height * dy	Accumulated Bucket State (Total Area)
Step 1	0.00	0.5 * 0.00 = 0.00	0.00 * 0.5 = 0.00	0.00 (Base Initialization State)
Step 2	0.50	0.5 * 0.50 = 0.25	0.25 * 0.5 = 0.125	0.00 + 0.125 = 0.125
Step 3	1.00	0.5 * 1.00 = 0.50	0.50 * 0.5 = 0.25	0.125 + 0.25 = 0.375
Step 4	1.50	0.5 * 1.50 = 0.75	0.75 * 0.5 = 0.375	0.375 + 0.375 = 0.75 **[Integration Phase Lock]**

D. Library Automation Shortcut Engine (SciPy Quadrature Automation)

```
from scipy.integrate import quad

def pdf_formula(y):
    return 0.5 * y

# --- SHORTCUT QUAD INTEGRATION ENGINE LOGIC ---
# quad auto extracts structural algorithm pointers without loops
total_area, error = quad(pdf_formula, 0, 2)
print(f"Automated Total Area: {total_area:.2f}")

target_probability, error = quad(pdf_formula, 1, 2)
print(f"Automated Target Window P(1 <= Y <= 2): {target_probability * 100:.2f}%")
```

📊 Architectural Quadrature `quad` & Error Core Analytics:

1. Functional Variable Tracking: SciPy Automated `quad` controller engine runtime function variable reference data array pointer mapping khud evaluate karta hai. Hume function block execution sequence me manual parameters explicitly stream out karne ki zarurat nahi hoti.

2. Absolute Error Estimate Framework Deep Logic: SciPy `quad` function structural output pattern processing state hamesha double pointer variable array stream data pass karta hai: `(calculated_result, error_estimate)`.

Computer ke paas real world parameter key matrix pre-built nahi hota, isiliye runtime memory segmentation calculation architecture parallel tracks par do algorithm mechanisms execute karti hai: **Rough Estimate Scale** and **Refined Matrix Step Track**.

In dono independent execution engines ke structural calculations end points me jo absolute numerical numeric variant distance benchmark gap bankar system state me capture hota hai (jaise 1.11×10^{-14} yaani 0.0000000000000111), usi variance value scale benchmark delta accuracy estimate measurement report ko system automatic backup safety tracking device container variable space jise hum `error` variable ka name dete hain, usme dump (save) kar deta hai.

PREPARED ESPECIALLY FOR:

Abu Fraz Abbas

Continuous Skill Development Plan — Milestone Day 11